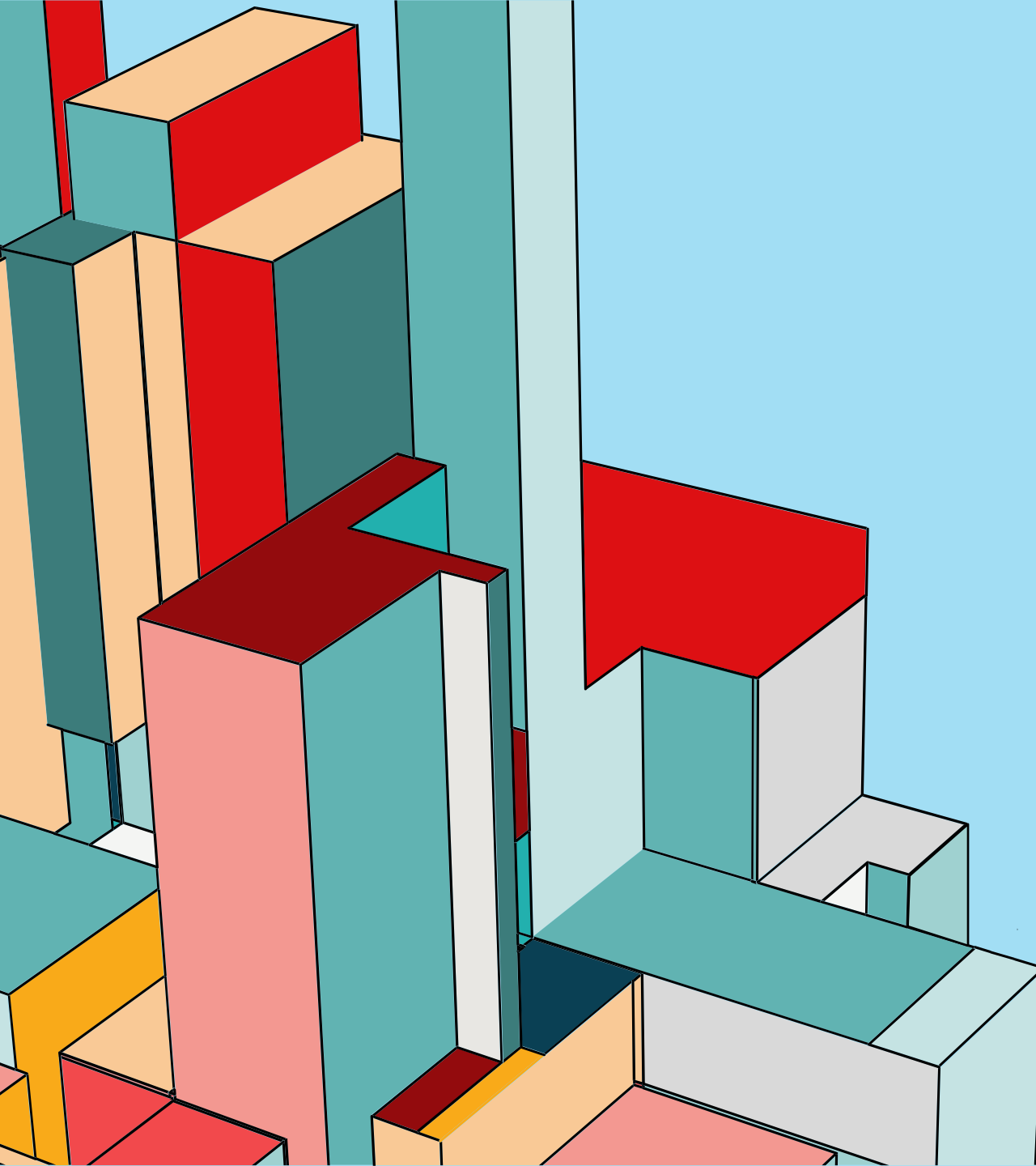


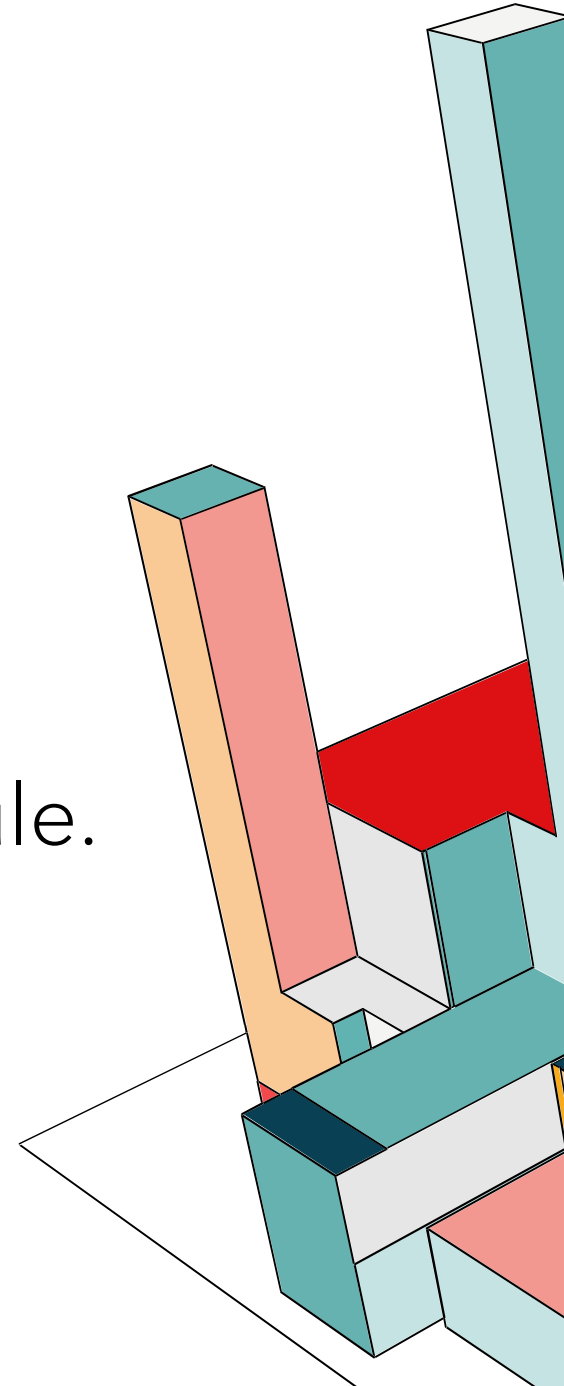


LECTURE 9 GREEDY ALGORITHMS

CSIS 2300 Introduction to
Algorithms, Spring 2026
Dr. Dan DiTursi

ANNOUNCEMENTS

- Exam grades should be available before next class.
- HW 5 posted later today.
- Office hours will also follow a Monday schedule.
- WASH. YOUR. HANDS!



MAKING CHANGE

US coinage comes in quarters (25 cents), dimes (10 cents), nickels (5 cents), and pennies* (1 cent).

(Yes, I am aware we're stopping with pennies AND I know about half-dollar and dollar coins. Just roll with me here.)

You are asked to generate n cents. What algorithm do you use to select the coins?

What if you are asked to do so using the minimum number of coins? Does your approach still work?

A GREEDY APPROACH

The standard approach to making change is simple: Use as many of the largest coin as possible, and then as many as possible of the next largest, and so on.

$$68¢ = Q(43¢ \text{ left}) + Q(18¢ \text{ left}) + D(8¢ \text{ left}) + N(3¢ \text{ left}) + P + P + P$$

This is a simple example of a *greedy algorithm*, which is one that always acts to maximize *short-term* gain; greedy algorithms have no lookahead.

GREEDY ALGORITHM – FORMAL DEFINITION

Greedy algorithms, like many of the algorithms we have already discussed, build up a solution by expanding a partially constructed solution one step at a time until the result is complete.

What makes a process *greedy* is that each step is:

- Feasible (allowed by the problem constraints)
- Irrevocable (can't change your mind later)
- Locally optimal (it's the best possible next step, regardless of what future steps look like)

TRAITS OF GREEDY ALGORITHMS

Like brute force, greedy algorithms are generally simple in design and easy to come up with.

When a greedy algorithm works, it often executes quite quickly – there's almost never a need for lots of nested loops or heavy use of recursion.

These properties usually make it worthwhile to construct a greedy algorithm as a first step, but...

Unlike brute force, it is not common that a greedy algorithm produces a correct (i.e. optimal) solution.

MAKING CHANGE, REDUX

Imagine that instead of abolishing pennies, we get rid of nickels, so our available coins are 25¢, 10¢, and 1¢.

You are asked to generate n cents using a minimum number of coins. What algorithm do you use to select the coins?

Find a value for n where the greedy approach does not give an optimal answer.

SUBMITTY POLL #1

Find a number of cents n for which the greedy approach does not give the correct answer.



GREED IS... NOT SO GOOD.

Imagine that instead of abolishing pennies, we get rid of nickels, so our available coins are 25¢, 10¢, and 1¢.

As an easy example, how should we make 30¢?

A: Three dimes.

But the greedy algorithm will grab a quarter, and then it will be forced to take five pennies, for six total coins.

It should be unsurprising that an algorithm that has no lookahead fails to perform optimally in most cases – if anything, it's a bit surprising that it DOES work sometimes!

When a greedy algorithm works, this tells us something about the fundamental structure of the problem.

GREEDY ALGORITHMS REQUIRE PROOF

Anytime we design a greedy algorithm, we will need to do one of two things:

- Demonstrate that it is not optimal by providing a counterexample.
 - This can still be productive – many such algorithms provide useful approximations to the optimal solution.
- Prove that it always produces an optimal solution.

PROVING OPTIMALITY

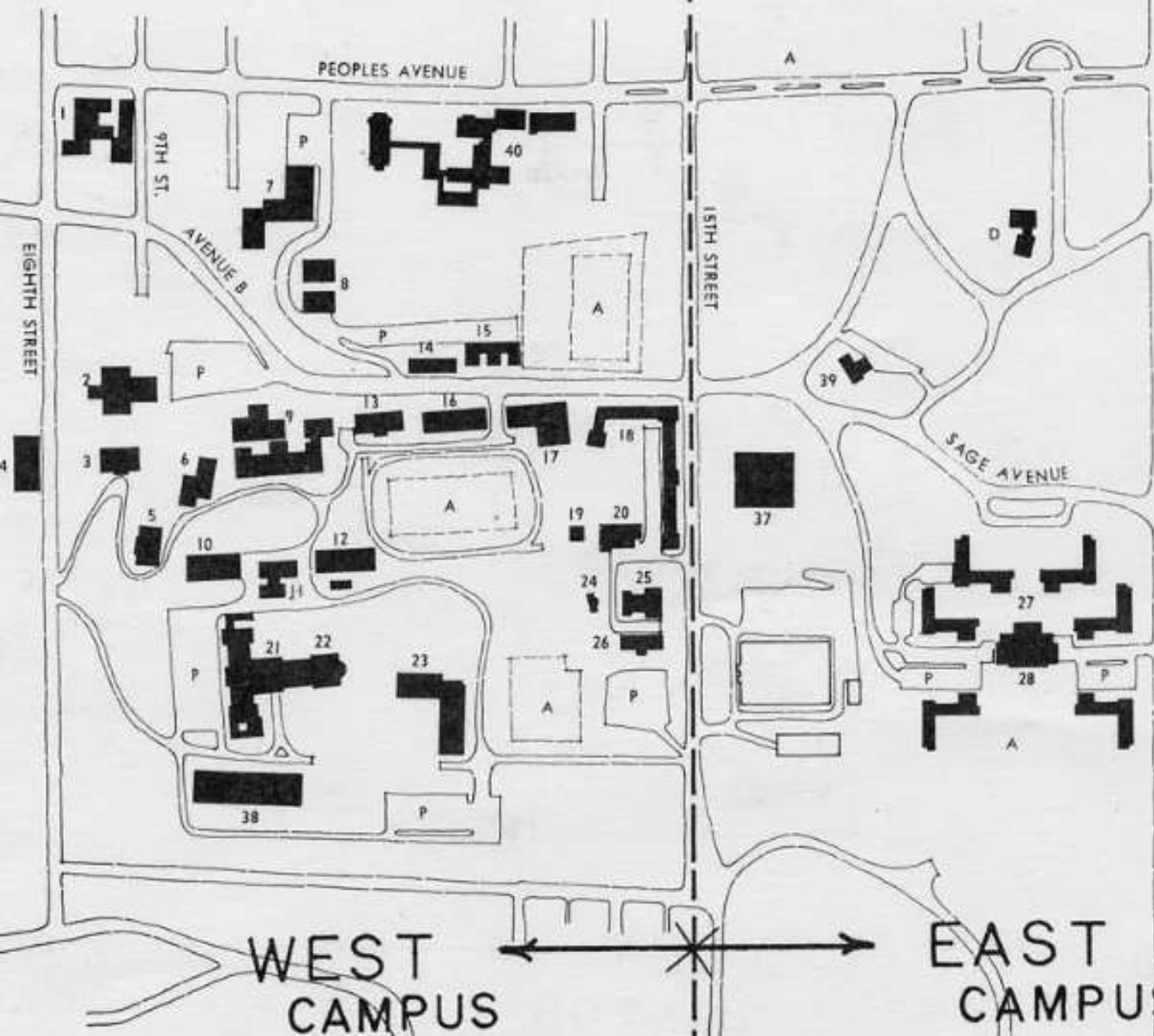
Proving that a greedy algorithm produces an optimal solution usually proceeds in one of two ways:

- “Wire-to-wire” or “gets ahead, stays ahead”: Show that at each stage, regardless of the partial solution constructed so far, there can never be a next step better than the one selected by greedy. Thus, no amount of clever “game planning” will allow other algorithms to catch up.
- An “exchange argument”: Show that any complete solution can be transformed into the greedy solution (by swapping one selection for another) without losing quality. Since you can do that with the optimal solution, the greedy solution must be at least as good as the optimal one.



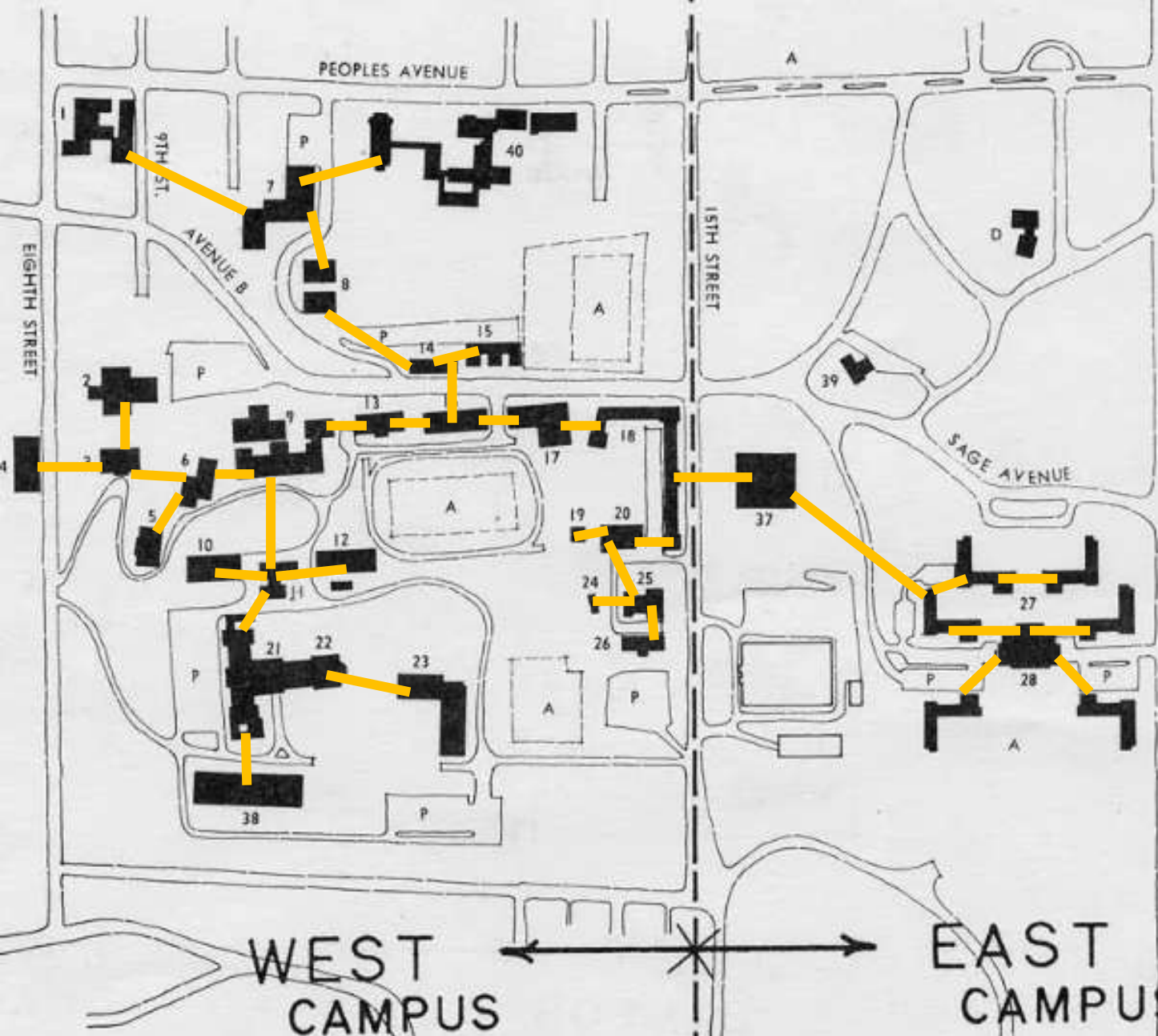
1967

THE CAMPUS OF
RENSSELAER
POLYTECHNIC INSTITUTE
TROY, NEW YORK 12181



Consider the problem of connecting up the steam system (or power, or Internet) to the buildings on campus. For cost reasons, we'd like to use a minimum amount of pipe.

Assume that we can connect any pair of buildings and each building needs only a single connection to the network. Which buildings should we connect?



You might come up with something like this, and it would probably be close to right.

But what algorithm should we use, in general, to find the optimal such network?

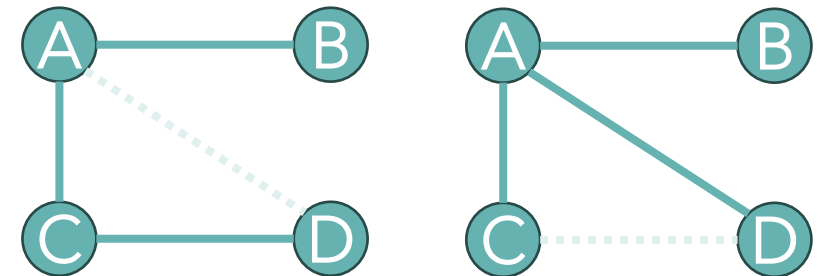
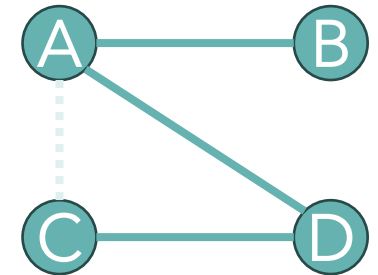
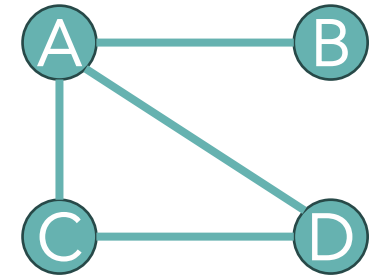
It depends on our data structure. As presented, this is a graph problem...

SPANNING TREES

Observe that the network structure we want here is a tree – that is, a connected graph with no cycles. (Why?)

When a tree includes every vertex of a graph G and uses only edges that are contained in G , it is called a *spanning tree* of G . In general, G can have many different spanning trees.

Note that only connected graphs have spanning trees.

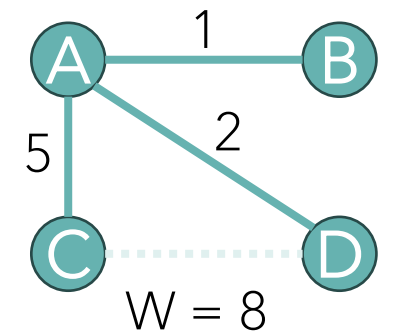
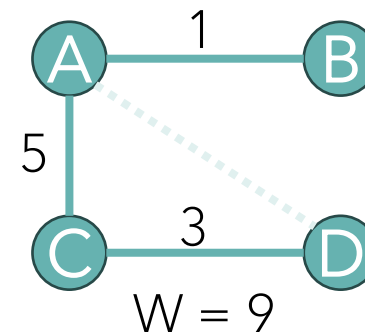
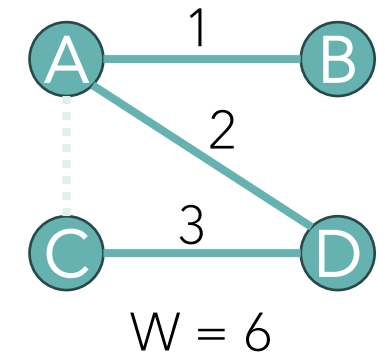
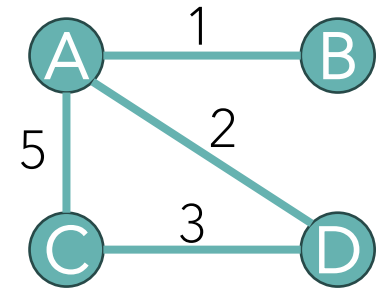


MINIMUM SPANNING TREES

Every spanning tree has $|V| - 1$ edges, and finding one is not hard.

The problem becomes interesting when the edges have weights, usually representing some kind of cost.

In that case, we can ask: Which spanning tree has the minimum total weight on its edges? This is the *minimum spanning tree (MST)* problem.



FINDING A MINIMUM SPANNING TREE

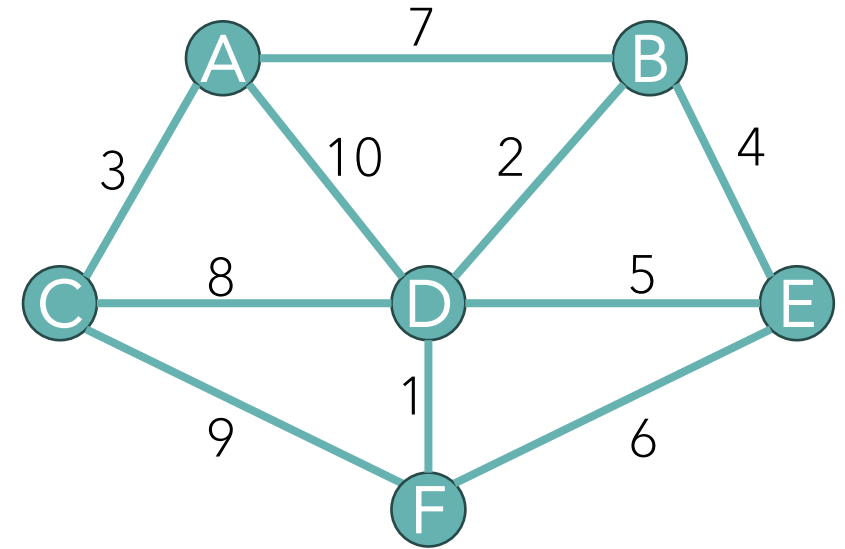
Q: Given a graph with V vertices and E edges, how many spanning trees are there?

A: This is not a trivial question! It depends heavily on the graph structure. One very naïve algorithm would be to check all subsets of the edges; this would be $O(2^E)$.

The number of spanning trees is also generally exponential in E , so a brute force approach to MST is out for all but the smallest graphs. Fortunately, there are better ways...

Do now: Try to design a greedy algorithm to solve MST. (You may assume that all of the edge weights are distinct.) A small graph is on the next slide for you to try your algorithms on.

FINDING A MINIMUM SPANNING TREE



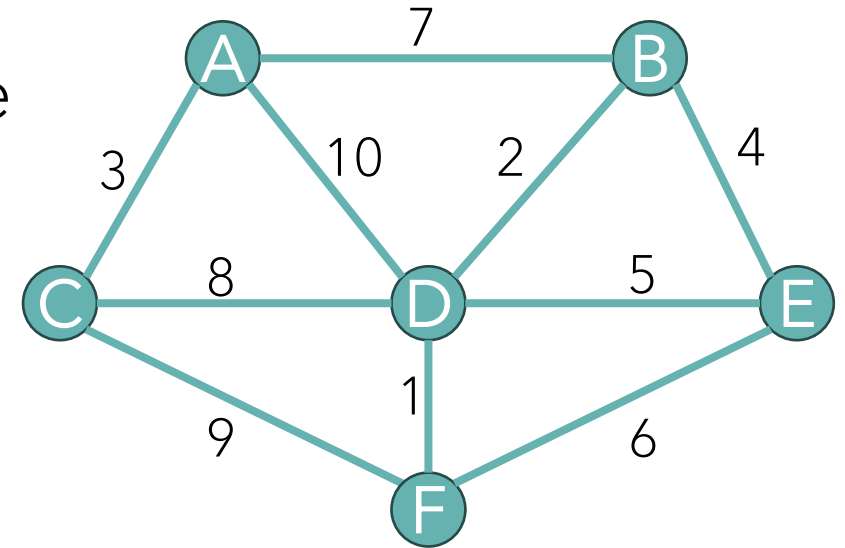
FINDING A MINIMUM SPANNING TREE

There are a variety of different ways to be “greedy” while building a spanning tree:

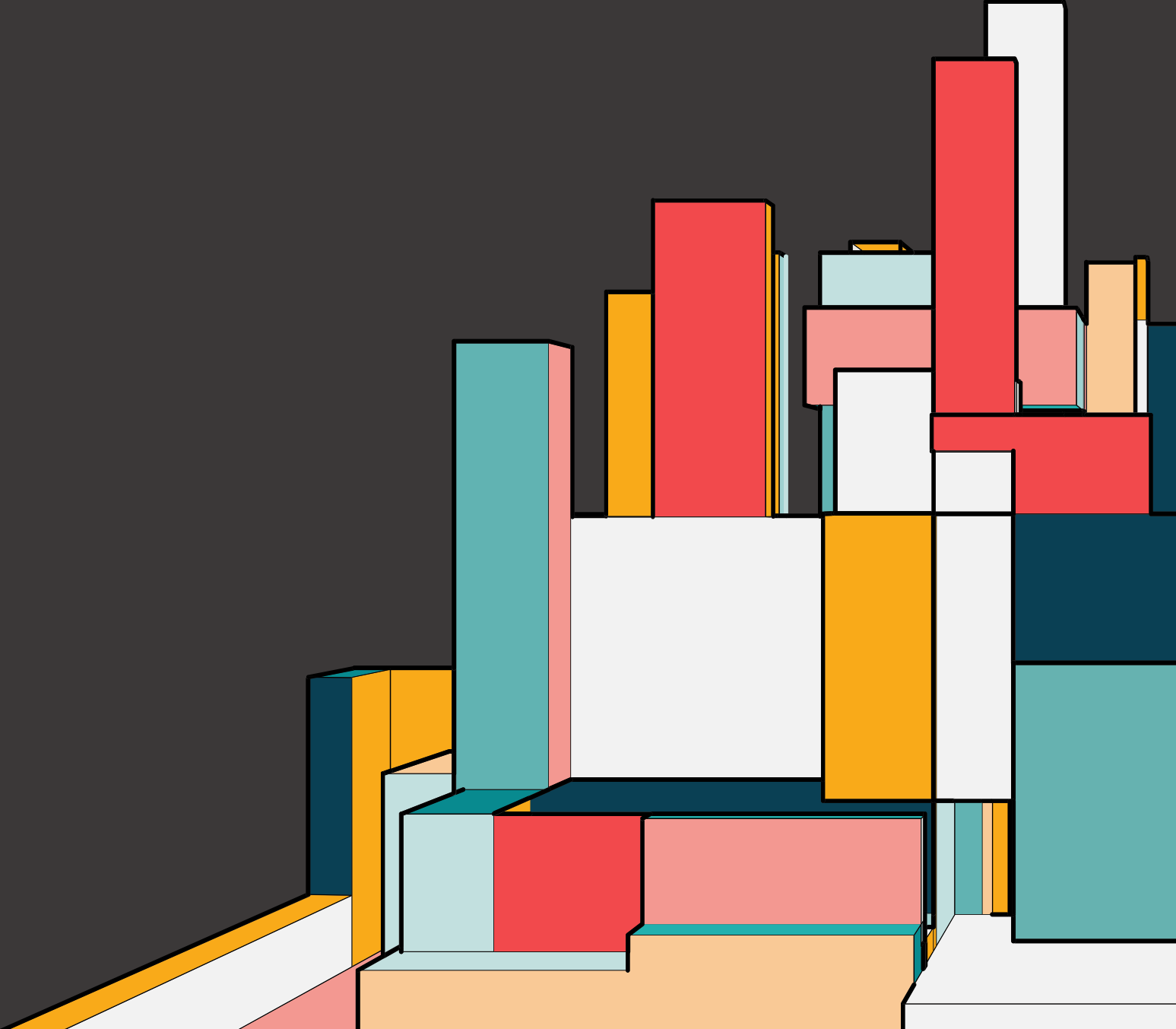
- Start from one node and grow a tree out from there by adding the cheapest node each time.
- Delete the most expensive edge that doesn't disconnect the graph.
- Add the cheapest non-redundant edge.

Which of these will work?

All of them!



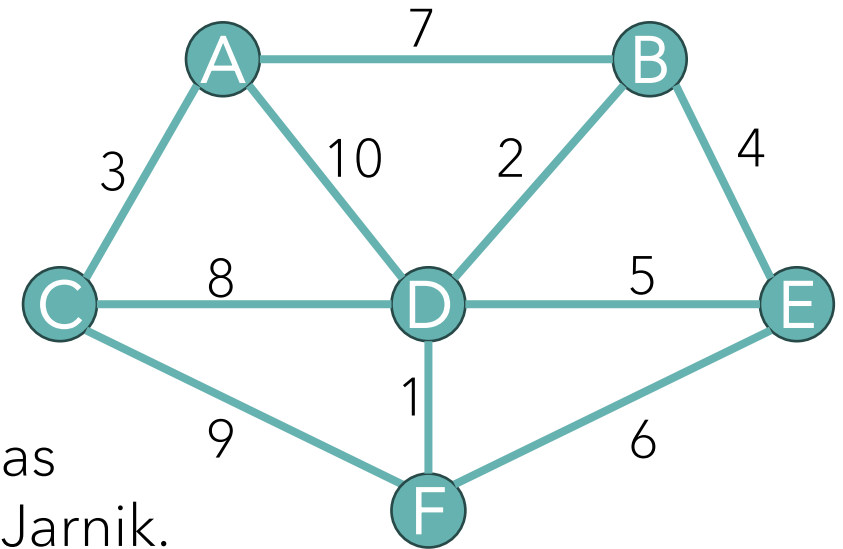
QUICK BREAK



PRIM'S ALGORITHM

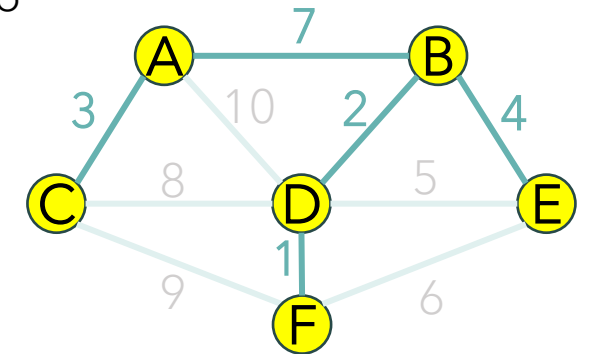
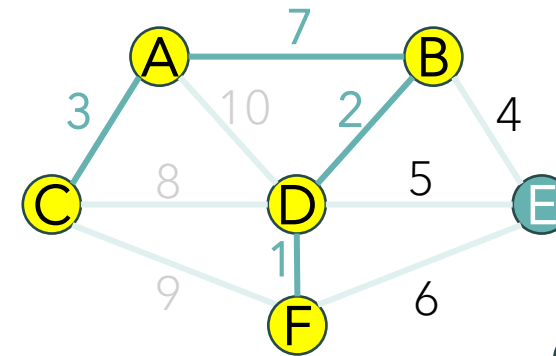
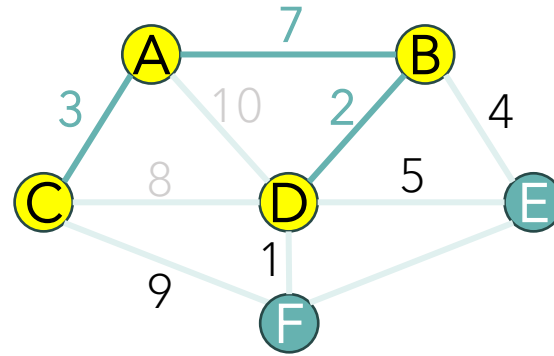
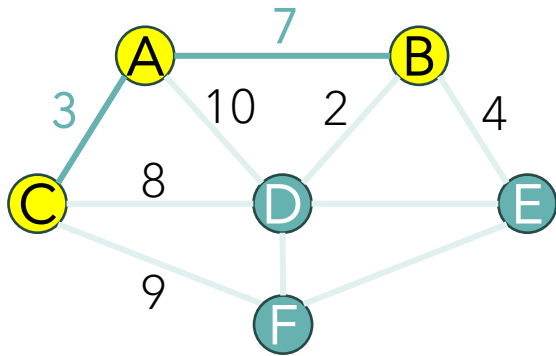
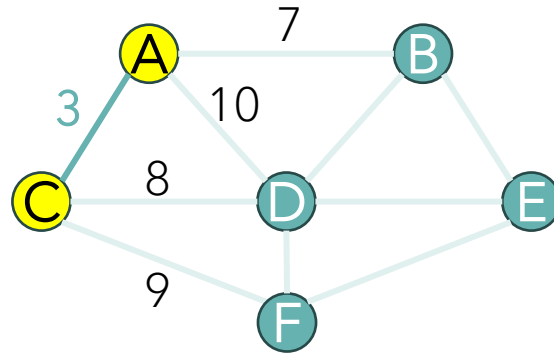
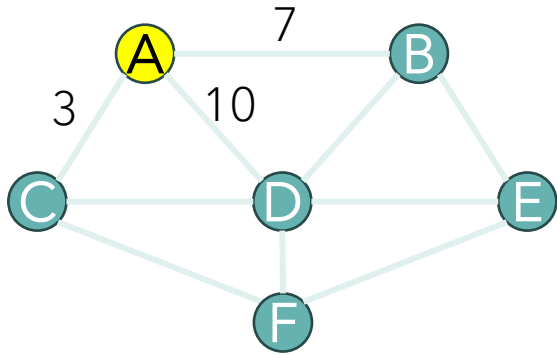
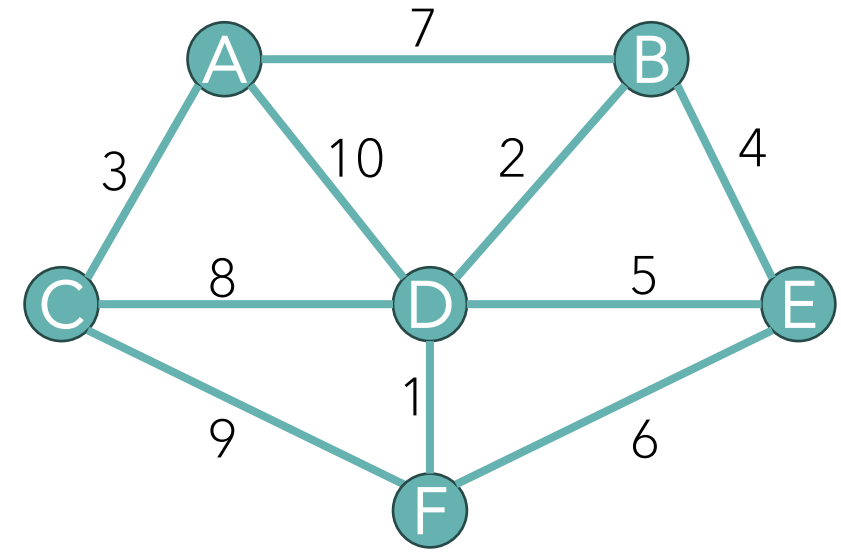
Perhaps the most intuitive approach is to start at some vertex and build up a tree from there. This is *Prim's algorithm*.

Unsurprisingly, Prim was not the first. In 1957, he independently recreated this approach, which was published in a Czech journal in 1930 by Vojtěch Jarník.



The core concept is that at every step, our partial solution is an MST for a subset of the vertices in the graph. The greedy step is to add the vertex to our tree that we can do most cheaply.

PRIM'S ALGORITHM



PRIM'S ALGORITHM

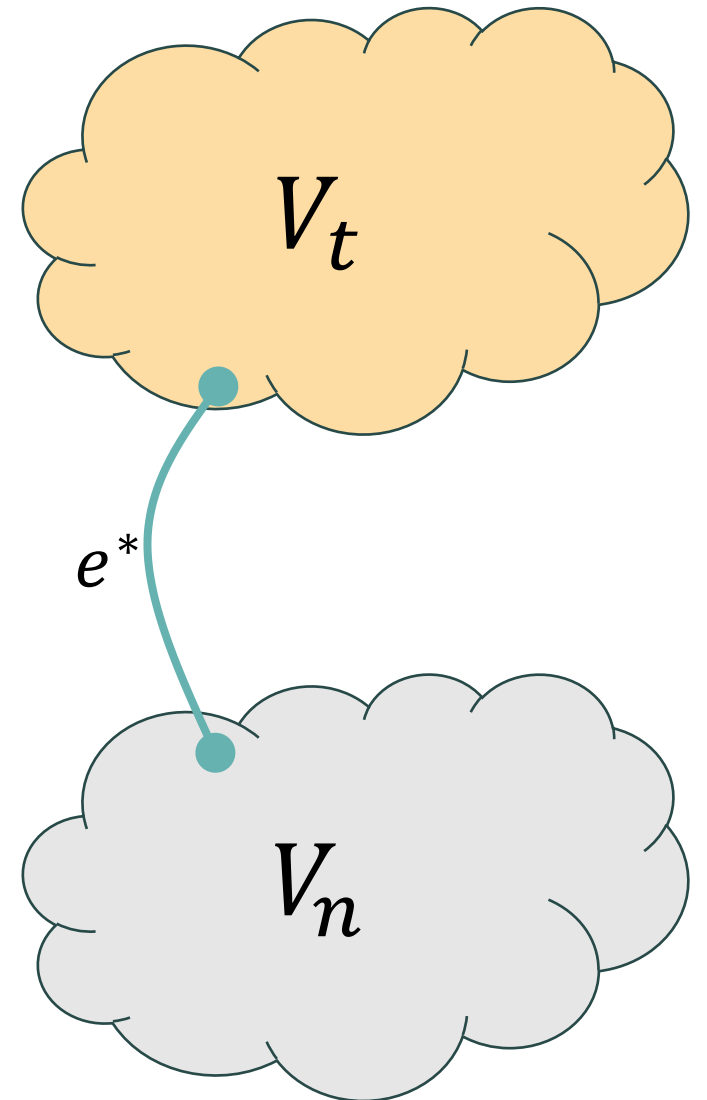
Throughout Prim's algorithm, we maintain a partition of the vertices into V_t , the vertices in our partial tree (the yellow ones) and V_n , the vertices not yet in our tree (the blue ones). E_t will be the set of edges that we use in our MST.

```
def Prim(G(V,E)):  
    Select arbitrary vertex and add it to  $V_t$   
    Add all other vertices to  $V_n$ ; let  $E_t = \emptyset$   
    do  $|V|-1$  times:  
        Find  $e^* = (v^*, u^*)$ , the minimum weight edge connecting  
            any  $v$  in  $V_t$  to any  $u$  in  $V_n$   
        Move  $u^*$  from  $V_n$  to  $V_t$ ; Add  $e^*$  to  $E_t$   
    return  $E_t$ 
```

OPTIMALITY OF PRIM'S ALGORITHM

To prove the Prim's algorithm really does produce the minimum spanning tree, we will use an "exchange argument" proof, which are inherently a sort of proof by contradiction.

Consider any edge e^* added by Prim's algorithm. By definition, it is the minimum weight edge connecting V_t and V_n (as constituted right before e^* is added).



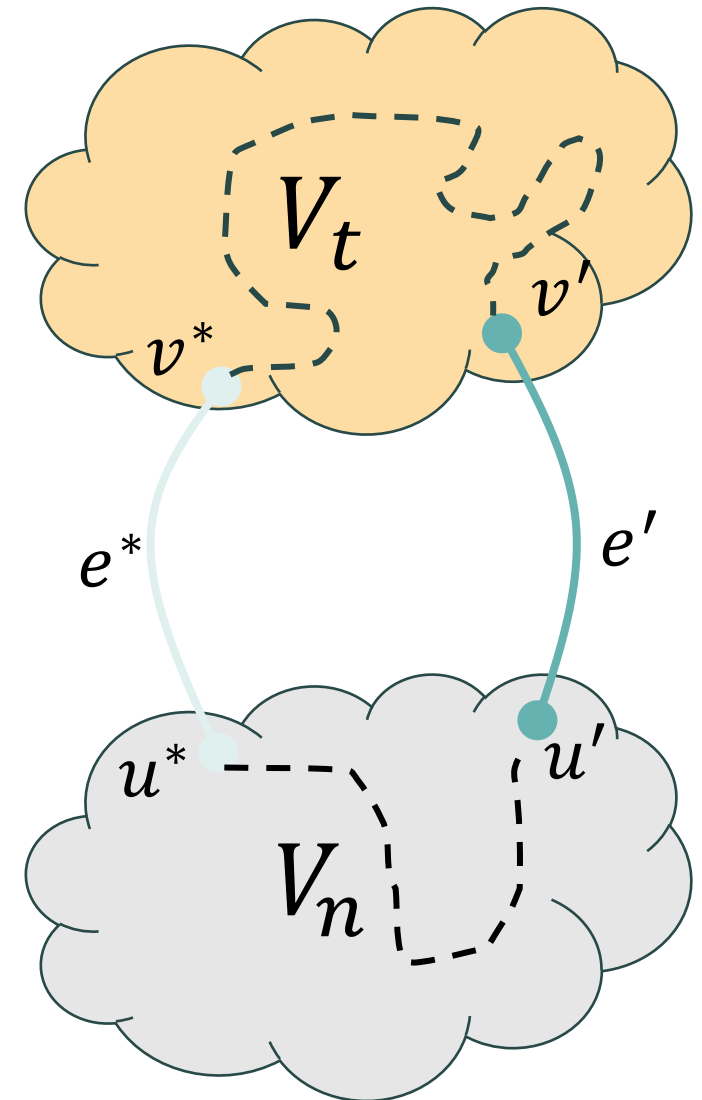
OPTIMALITY OF PRIM'S ALGORITHM

Now, assume for contradiction that there is an MST T' that doesn't use e^* .

Perforce, T' must use some other edge e' that connects V_t and V_n , since the overall MST must be connected.

Let v^*, u^* be the endpoints of e^* and v', u' be the endpoints of e' . (Note that one of these pairs might overlap; that's fine.)

It must be the case that v^* and v' are connected in T' ; ditto u^* and u' .



OPTIMALITY OF PRIM'S ALGORITHM

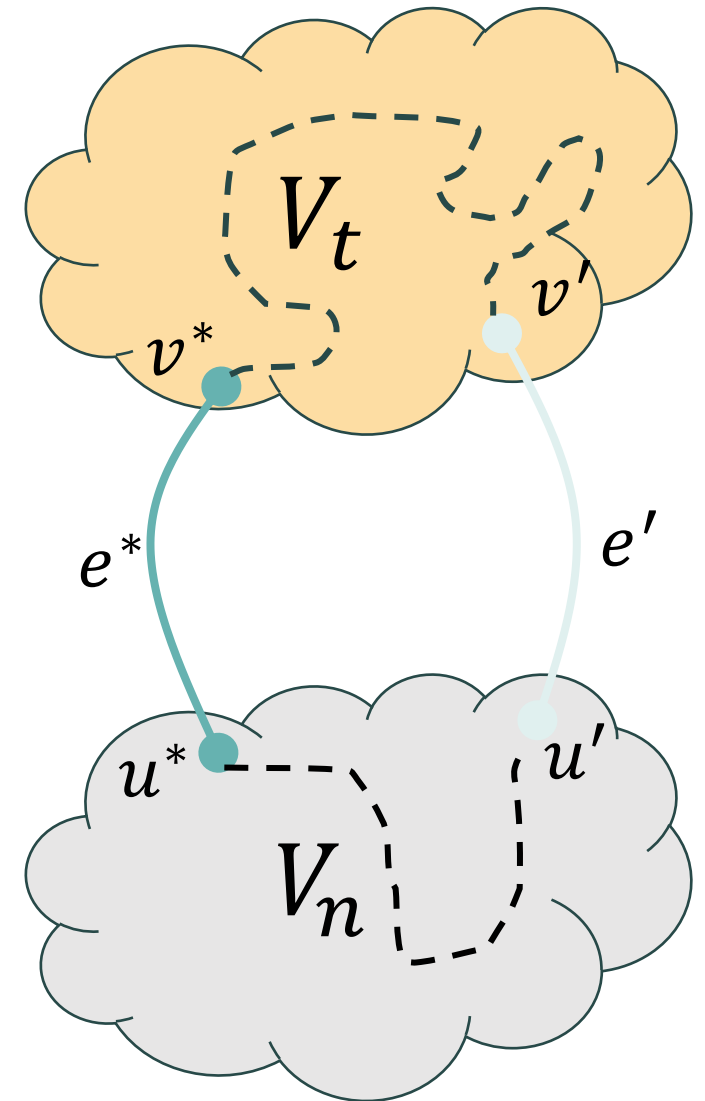
Therefore, from T' we can construct a new spanning tree $T^* = T' - \{e'\} \cup \{e^*\}$.

e^* was specifically chosen to be the lowest weight edge from V_t to V_n ; in particular, $w(e^*) < w(e')$.

Since all of the other edges are the same, $w(T^*) < w(T')$, a contradiction!

Therefore, every MST must use edge e^* .

Since e^* was an arbitrary edge selected by Prim, it must be that every edge selected by Prim is part of every MST. ■



PRIM'S ALGORITHM: IMPLEMENTATION DETAILS

```
def Prim(G(V,E)):
```

```
    Select arbitrary vertex and add it to  $V_t$ 
```

```
    Add all other vertices to  $V_n$ ; let  $E_t = \emptyset$ 
```

```
    do  $|V|-1$  times:
```

```
        Find  $e^* = (v^*, u^*)$ , the minimum weight edge connecting  
        any  $v$  in  $V_t$  to any  $u$  in  $V_n$ 
```

```
        Move  $u^*$  from  $V_n$  to  $V_t$ ; Add  $e^*$  to  $E_t$ 
```

```
    return  $E_t$ 
```

Feels like that step in red above is doing a LOT of work... and it is!

If we do that poorly, we could spend $O(V^2)$ time on that step, each time through the loop!



PRIORITY QUEUE TO THE RESCUE

In practice, we can handle it much faster using a *priority queue* – a data structure that keeps elements in sorted order by an assigned priority.

The usual priority queue implementation uses a structure called a *heap*, which we will get to later in the course.

For now, the key properties of our priority queue are that elements can be added (or have their priority changed) in $O(\log n)$ time (where n is the number of elements in the PQ) and that the highest-priority element can be retrieved in $O(1)$ time.

PRIM'S ALGORITHM REVISED

```
def Prim(G(V,E)):
    Let  $v = v_0$  and add it to  $V_t$ ; let  $E_t = \emptyset$ ; PQ starts empty
    do  $|V|-1$  times:
        for each neighbor  $u$  of  $v$ :
            if  $u$  is not already in  $V_t$ :
                if  $u$  is not in the PQ already:
                    add  $u$  to the PQ with priority  $w(v,u)$  and parent  $v$ 
                else: #  $u$  is already in the PQ
                    update priority and parent if better than previous
             $v = \text{top element in PQ}$ 
        add  $v$  to  $V_t$ ; add  $(v, \text{parent}(v))$  to  $E_t$ 
```

What is the runtime of this algorithm?

PRIM'S ALGORITHM REVISED

```
def Prim(G(V,E)):
```

```
    Let  $v = v_0$  and add it to  $V_t$ ; let  $E_t = \emptyset$ ; PQ starts empty
```

```
    do  $|V|-1$  times:
```

```
        for each neighbor  $u$  of  $v$ :
```

```
            if  $u$  is not already in  $V_t$ :
```

```
                if  $u$  is not in the PQ already:
```

```
                    add  $u$  to the PQ with priority  $w(v,u)$  and parent  $v$ 
```

```
                else: #  $u$  is already in the PQ
```

```
                    update priority and parent if better than previous
```

```
             $v =$  top element in PQ
```

```
            add  $v$  to  $V_t$ ; add  $(v, \text{parent}(v))$  to  $E_t$ 
```

Executes twice per edge

Needs
 $O(\log V)$
time

What is the runtime of this algorithm? $O(E \log V)$

MST APPROACH #2: KRUSKAL'S ALGORITHM

As previously mentioned, there are multiple greedy approaches to generating a minimum spanning tree – and astonishingly, ALL of them work!

The second technique we'll examine is called Kruskal's algorithm. The intuition is perhaps even simpler than Prim's: Just add the cheapest edge you can every time, as long as the resulting structure is still a tree!

KRUSKAL'S ALGORITHM PSEUDOCODE

```
def Kruskal(G(V,E)):  
    Sort E by weight:  $[e_1, e_2, \dots]$   
     $E_t = \emptyset$ ; count = 0; i = 0  
    while count <  $|V|-1$ :  
        k = k+1  
        if  $E_t \cup \{e_i\}$  has no cycle:  
            add  $e_i$  to  $E_t$  and increment count
```

As with Prim's algorithm, we have a step that is doing some heavy lifting here – what is it?

KRUSKAL'S ALGORITHM PSEUDOCODE

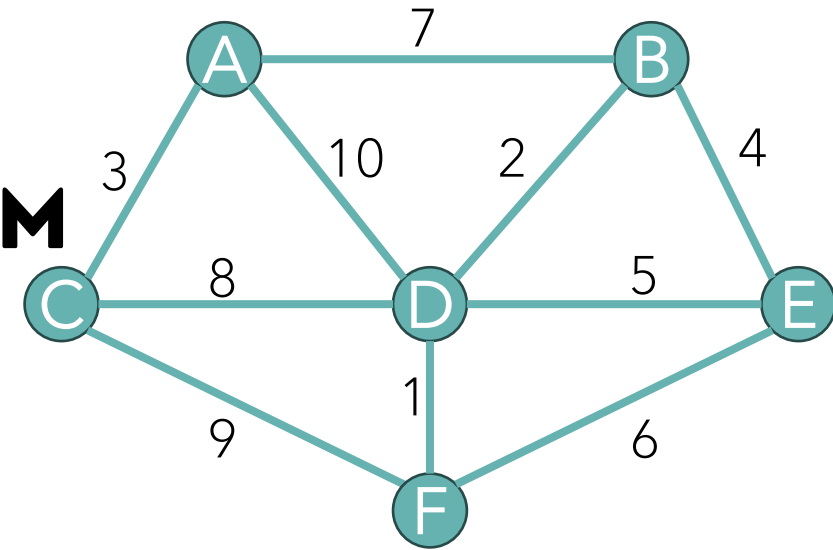
```
def Kruskal(G(V,E)):  
    Sort E by weight:  $[e_1, e_2, \dots]$   
     $E_t = \emptyset$ ; count = 0; i = 0  
    while count <  $|V|-1$ :  
        k = k+1  
        if  $E_t \cup \{e_i\}$  has no cycle:  
            add  $e_i$  to  $E_t$  and increment count
```

Determining if the edge produces a cycle is non-trivial. Fortunately, there is a fast approach that relies on a *union-find* algorithm, which we will not go into the details of. It makes the entire loop faster than the sort, so the runtime here is $O(E \log E)$.

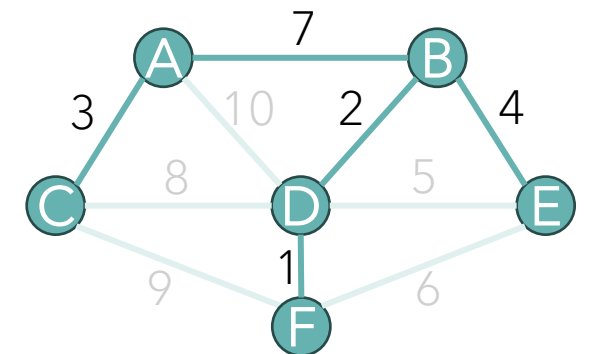
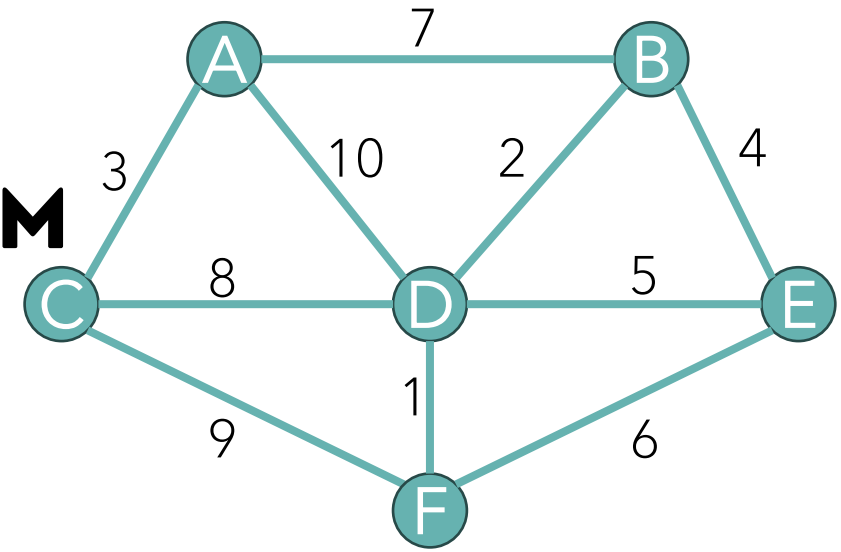
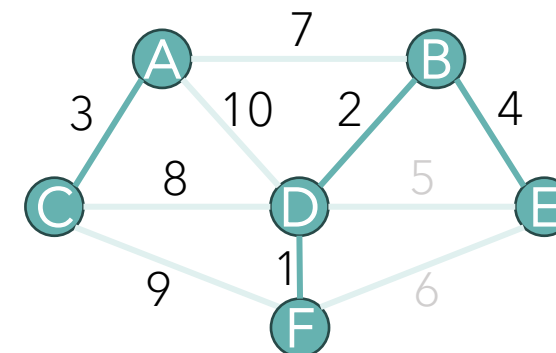
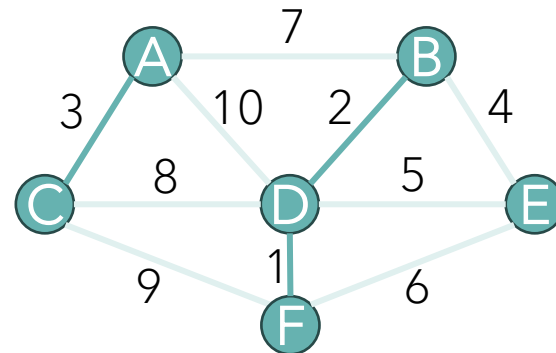
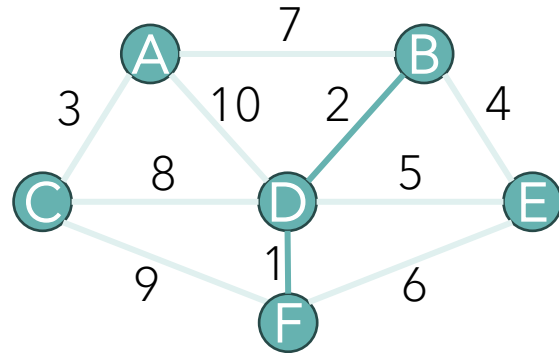
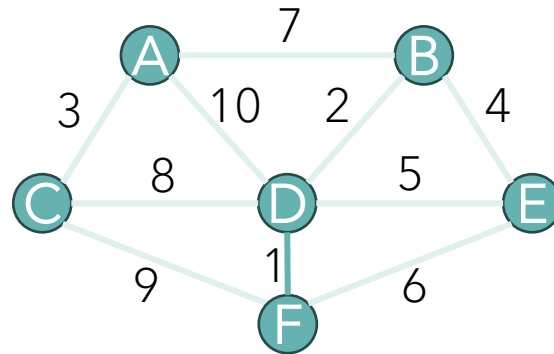
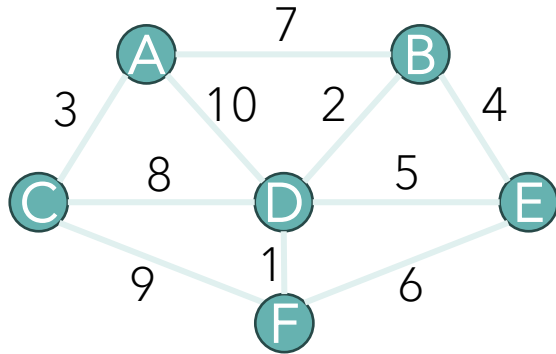
EXECUTING KRUSKAL'S ALGORITHM

Using the same graph as before, execute Kruskal's algorithm now. Determine which edge is the last one added to the MST, and respond to the poll question with that answer.

(Write it as two letters: BC would mean the (non-existent) edge between vertices B & C.)



EXECUTING KRUSKAL'S ALGORITHM



BEFORE YOU LEAVE



PRIM'S ALGORITHM

